实验摘要：

1、 用 Matlab、python 等编程语言实现题目给出的卷积操作，绘制出图形。
2、 用 Matlab、python 等编程语言求解微分方程系统的单位冲击响应、单位阶跃响应，深入理解单位冲击响应和单位阶跃响应。
3、 用 Matlab、pthon 等编程语言实现高斯白噪声并求自相关函数。
4、 用 Matlab、python 等编程语言观察傅里叶级数相关形状。
5、提高学生应用编程语言解决信号问题的能力。

实验题目：

1. 利用MATLAB求下列函数的卷积，并绘制出图形
   (1) $f_1(t)=\varepsilon(t)-\varepsilon(t-1)$， $f_2(t)=2t[\varepsilon(t)-\varepsilon(t-1)]$
   (2) $f_1(t)=\cos(30t)g_5(t)$， $f_2(t)=\varepsilon(t)-\varepsilon(t-4)$
   参考函数：conv( )

2. 某系统满足的微分方程为
$$y''(t)+4y'(t)+3y(t)=2f'(t)+f(t)$$
   (1)利用MATLAB求系统的单位冲击响应，并绘出图形
   (2)利用MATLAB求系统的单位阶跃响应，并绘出图形
   (3)利用MATLAB求系统对信号 $f(t)=4\sin(2\pi t)\varepsilon(t)$ 的响应，并绘出图形
   参考函数：tf( )，impulse( )，step()，lsim( )，conv( )

3. 利用MATLAB产生高斯白噪声，绘出图形，并求其自相关函数，绘出图形。
   参考函数：randn ()，wgn( )，xcorr( )，autocorr( )

4. 关于傅里叶级数，用MATLAB或者Python进行以下实验，回答问题并给出实验过程中产生的结果图。
   (1)信号 $f(t)$ 的傅里叶级数为 $\sum_{1}^{\infty}\frac{\sin nt}{n}$，代入数字去逼近或者用解析法分析，估计 $f(t)$ 的形式。
   (2)写出你估计出的 $f(t)$ 的傅里叶级数，与上式对比，说明它的谐波和正余弦分量的情况。
   (3)取 $N=50,100,200,.....$ 画出 $f_N(t)=\sum_{n=1}^{N}\frac{\sin nt}{n}$，当 $N\to\infty$ 时，判断这个部分和与 $f(t)$ 的区别。
   (4) 同样，取 $N=50,100,200,.....$ 画出 $F_N(t)=\frac{f_1(t)+f_2(t)+f_3(t)+...+f_N(t)}{N}$，和上面的图对比，分析他们之间的不同。

实验内容：

一、实验基本原理及步骤

1-（1）

先设计一个函数实现阶跃函数，用 f1 和 f2 表示出两个待求卷积的函数，调用 numpy 里面的 convolve 库来是实现两个函数的卷积操作，最后画出结果图。

具体实验代码如下：

```
def step_function(x):
    if x < 0:
        return 0
    else:
        return 1


t = np.linspace(-10, 10, 1000)
f1 = np.array([step_function(i) - step_function(i - 1) for i in t])
f2 = np.array([2 * i * (step_function(i) - step_function(i - 1)) for i in t])
f2_flipped = np.flipud(f2)
conv_result = np.convolve(f1, f2_flipped, mode='same')

plt.figure(figsize=(10, 10))
plt.plot(t, conv_result, label='conv_result')
plt.legend()
plt.xlabel('t')
plt.ylabel('F')
plt.title('1-(1)')
plt.grid()
plt.show()
```

1-（2）

在第一问的基础上，需要加上底边长为 5 的门函数，调用阶跃函数和门函数实现待卷积的两个函数 f1 和 f2，同样调用 numpy 库里面的 convolve 函数来实现两个函数的卷积操作。

具体实验代码如下：

```
def step_function(x):
    if x < 0:
        return 0
    else:
        return 1


def g_5_function(x):
    if x > 2.5 or x < -2.5:
        return 0
    else:
        return 1


t = np.linspace(-10, 10, 1000)
f1 = np.array([np.cos(30 * i) * g_5_function(i) for i in t])
f2 = np.array([(step_function(i) - step_function(i - 4)) for i in t])
f2_flipped = np.flipud(f2)
conv_result = np.convolve(f1, f2_flipped, mode='same')

plt.figure(figsize=(10, 10))
plt.plot(t, conv_result, label='conv_result')
plt.legend()
plt.xlabel('t')
plt.ylabel('F')
plt.title('1-(2)')
```

```
plt.grid()
plt.show()
```

2-（1）（2）

首先需要调用 scipy.signal 信号库里面的 lti 来将题目给出的微分方程系统表示出来，第 1）2）问都可以直接调用该库里面的 step 和 impulse 来直接实现对阶跃响应和冲击响应的求解，最后画出图即可。

具体实验代码如下：

```
from scipy.signal import lti, step, impulse
import matplotlib.pyplot as plt

sys = lti([2, 1], [1, 4, 3])  # 方程里的系数 由高次幂到低次幂
st, sy = step(sys)
it, iy = impulse(sys)
plt.plot(st, sy, label='step')
plt.plot(it, iy, label='impluse')
plt.legend()
plt.title('2-(1)(2)')
plt.grid()
plt.show()
```

2-（3）

同样需要先实现微分方程，然后要对输入函数表示出来，调用库函数 lsim 来模拟系统对输入信号的响应，最后画出图即可。

具体实验代码如下：

```
import numpy as np
from scipy.signal import lti, lsim
import matplotlib.pyplot as plt


def step_function(x):
    if x < 0:
        return 0
    else:
        return 1


sys = lti([2, 1], [1, 4, 3])  # 方程里的系数 由高次幂到低次幂

t = np.linspace(0, 10, 1000)
f1 = np.array([4 * np.sin(2 * np.pi * i) * step_function(i) for i in t])
# 模拟系统对输入信号的响应
t, response, _ = lsim(sys, f1, t)

plt.plot(t, response, label='output')
plt.legend()
plt.title('2-(3)')
plt.grid()
plt.show()
```

3

需要了解高斯白噪声信号是啥，调用 numpy 里面的随机生成数的库来生成高斯白噪声，画出图形，然后调用 correlate 库来对生成的高斯白噪声求解自相关函数。

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# 定义参数
num_samples = 1000
mean = 0
std_dev = 1

# 生成高斯白噪声
white_noise = np.random.randn(num_samples) * std_dev

# 计算自相关函数
autocorr = np.correlate(white_noise, white_noise, mode='full')

# 绘制结果
plt.figure(figsize=(7, 7))
plt.plot(white_noise)
plt.figure(figsize=(7, 7))
plt.plot(autocorr, color='r')
plt.xlabel('T')
plt.ylabel('y')
plt.title('Gaussian White Noise')
plt.grid(True)
plt.show()
```

4-（1）

先随机生成 0 到 4Π区间 1000 个点作为横坐标取值点，根据题目给出的级数形式，用循环来求解每一个对应 t 值的函数值，最后根据 t 和求解出来的函数值画出图像，观察即可。

具体实验代码如下:

```
import numpy as np
import matplotlib.pyplot as plt

n = 1000
f1 = []
t = np.linspace(0, np.pi*4, 1000)
for i in t:
    x = 0
    for j in range(1, n):
        x += np.sin(j*i) / j
    f1.append(x)
plt.figure(figsize=(7, 7))
plt.plot(t, f1, label='jishu')
plt.legend()
plt.xlabel('t')
plt.ylabel('F')
plt.title('4-(1)')
plt.grid()
plt.show()
```

4-（2）

跟据高数的基本知识可以知道 f（t）的原函数是对数函数，对对数函数进行傅里叶展开，观察展开项的谐波和正余弦分量情况。

4-(3)

对于 N 取不同值，观察 f（t）图像，实际上就是将第一问中 N=1000，切换成 N=50、100、200、300 来进行实验画图，最终将四张图作为子图放在一起来看不同 N 对 f（t）逼近效果

具体实验代码如下

```
import numpy as np
import matplotlib.pyplot as plt

# 定义时间 t 的范围
t = np.linspace(0, np.pi * 4, 1000)
```

```
# 设置子图的行数和列数
n_rows = 2
n_cols = 2

# 创建子图的网格
fig, axes = plt.subplots(n_rows, n_cols, figsize=(8, 8))

# 计算每个子图对应的 N 值
N_values = [50, 100, 200, 300]

# 遍历每个子图
for ax, N in zip(axes.flatten(), N_values):
    f1 = []
    for i in t:
        x = 0
        for j in range(1, N):
            x += np.sin(j * i) / j
        f1.append(x)
    # 在当前子图中绘制函数图像
    ax.plot(t, f1, label=f'N={N}')
    ax.legend()
    ax.set_title(f'N={N}')
    ax.grid(True)

# 设置子图之间的间距
plt.tight_layout()

# 显示图形
plt.show()
```

4-(4)

这个实际上就是实现 F（t）的函数，主要涉及到变成上循环的实现。

具体实验代码如下

```
import numpy as np
import matplotlib.pyplot as plt

# 定义时间 t 的范围
t = np.linspace(0, np.pi * 4, 100)

# 设置子图的行数和列数
n_rows = 2
n_cols = 2

# 创建子图的网格
fig, axes = plt.subplots(n_rows, n_cols, figsize=(8, 8))

# 计算每个子图对应的 N 值
N_values = [50, 100, 200, 300]

# 遍历每个子图
for ax, N in zip(axes.flatten(), N_values):
    f2 = []
    for i in t:
        f1 = []
        for n in range(1, N+1):
            x = 0
            for j in range(1, n):
                x += np.sin(j * i) / j
            f1.append(x)
```

```
        f2.append(sum(f1) / N)
    # 在当前子图中绘制函数图像
    ax.plot(t, f2, label=f'N={N}')
    ax.legend()
    ax.set_title(f'N={N}')
    ax.grid(True)

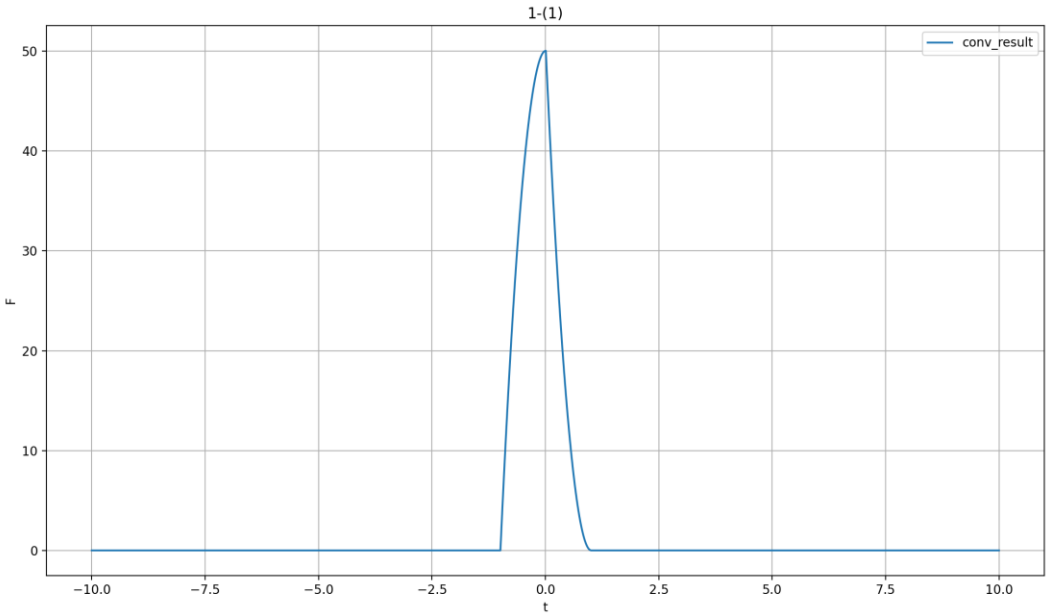
# 设置子图之间的间距
plt.tight_layout()

# 显示图形
plt.show()
```

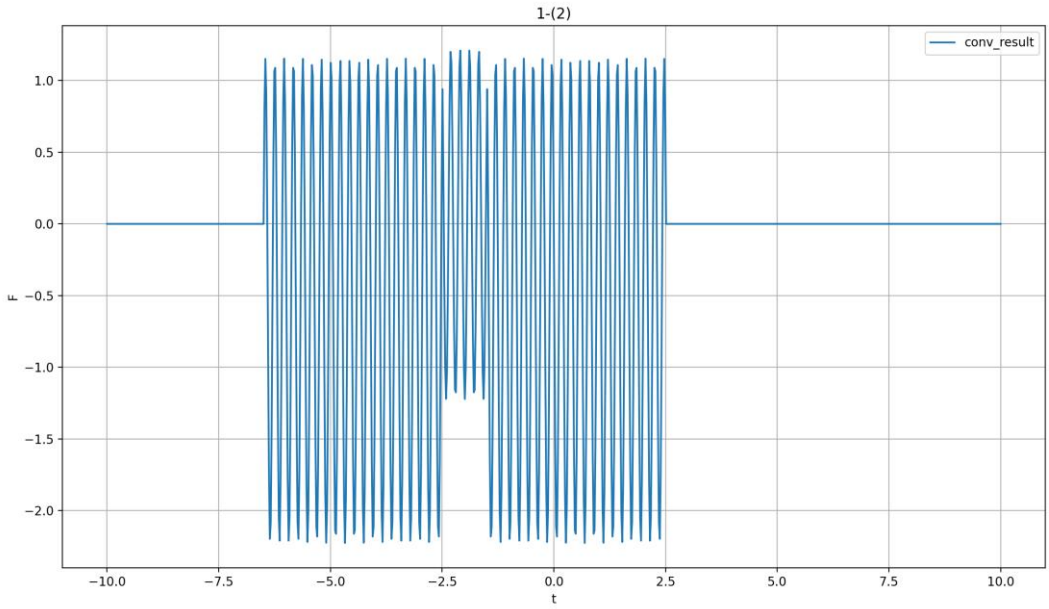
二、实验结果

实验最终绘制的图形如下一一所示：

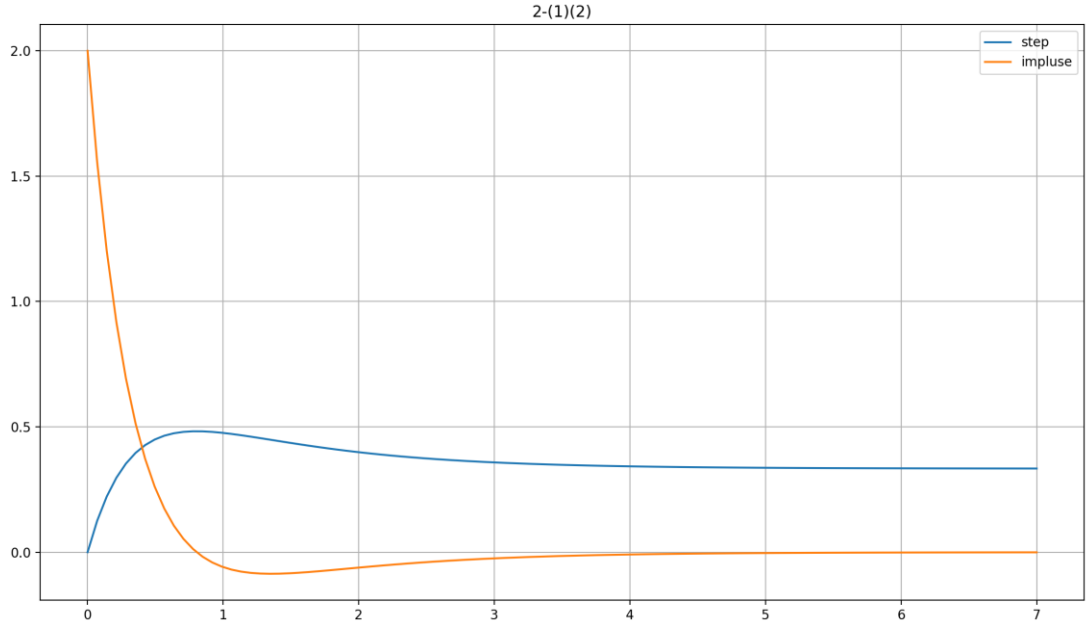
1-（1）



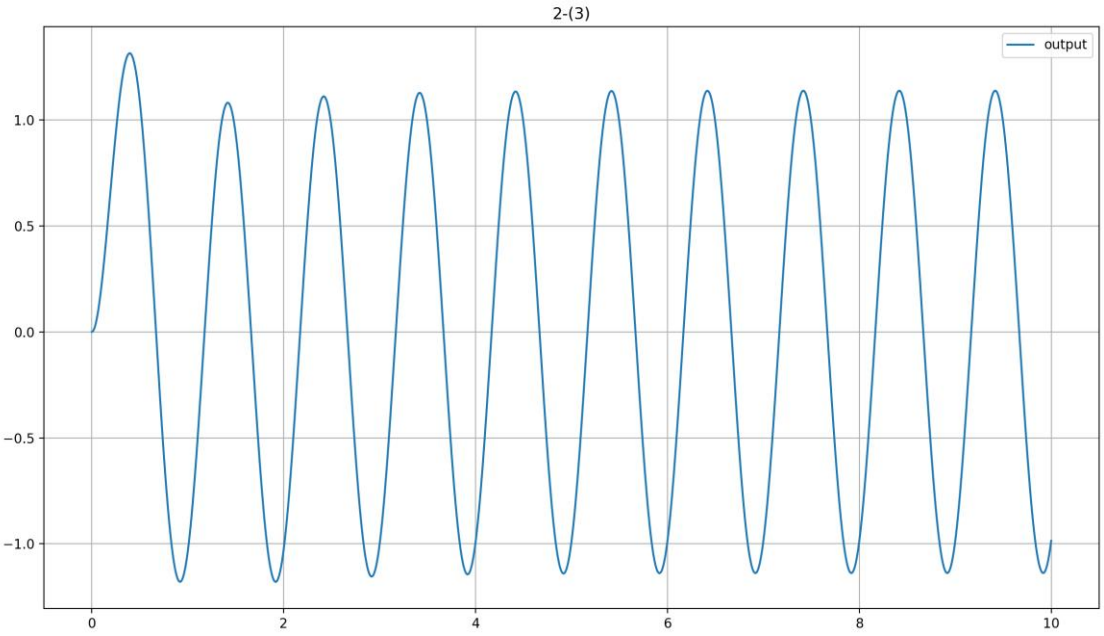
1-（2）



2-（1）（2）



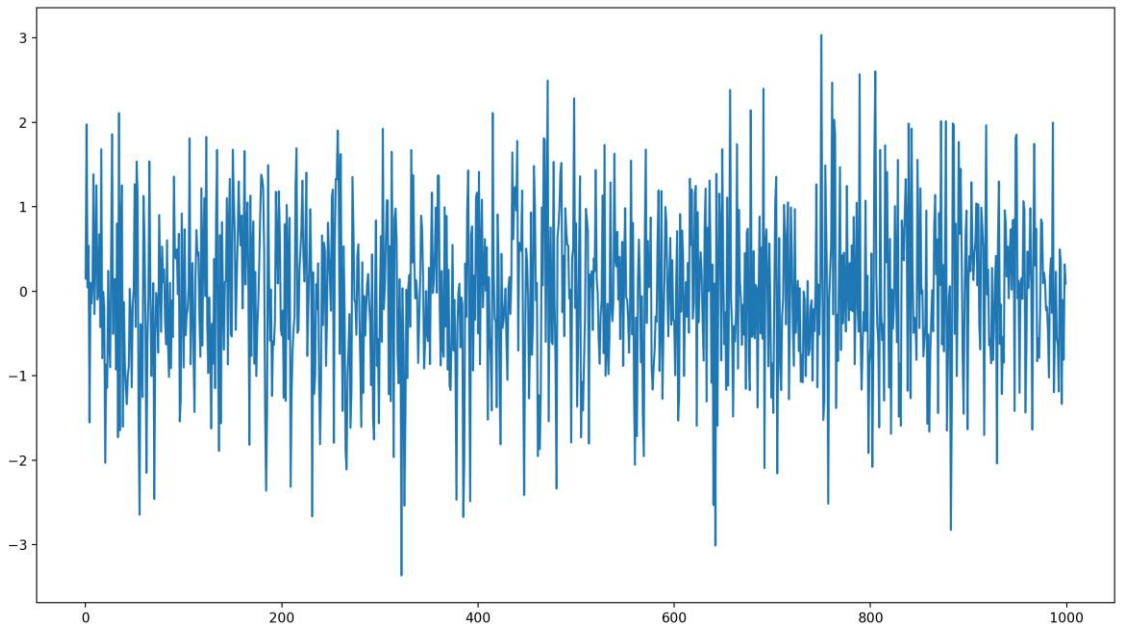
2-（3）



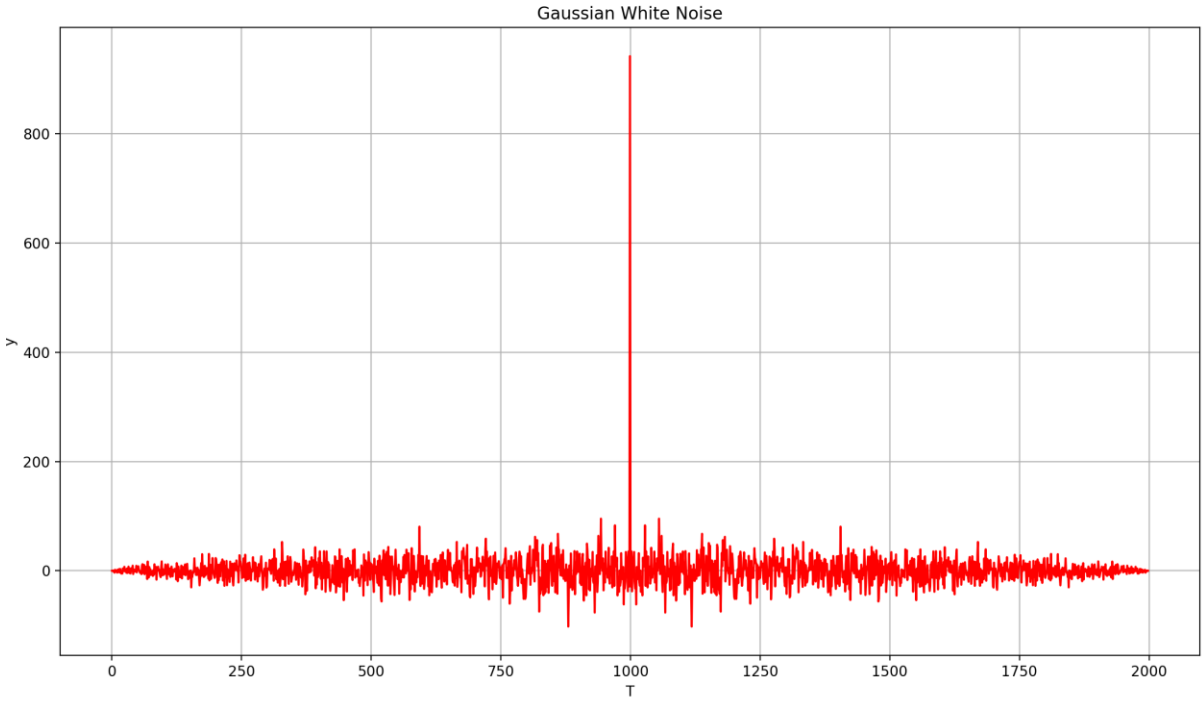
3

高斯白噪声图像

自相关函数

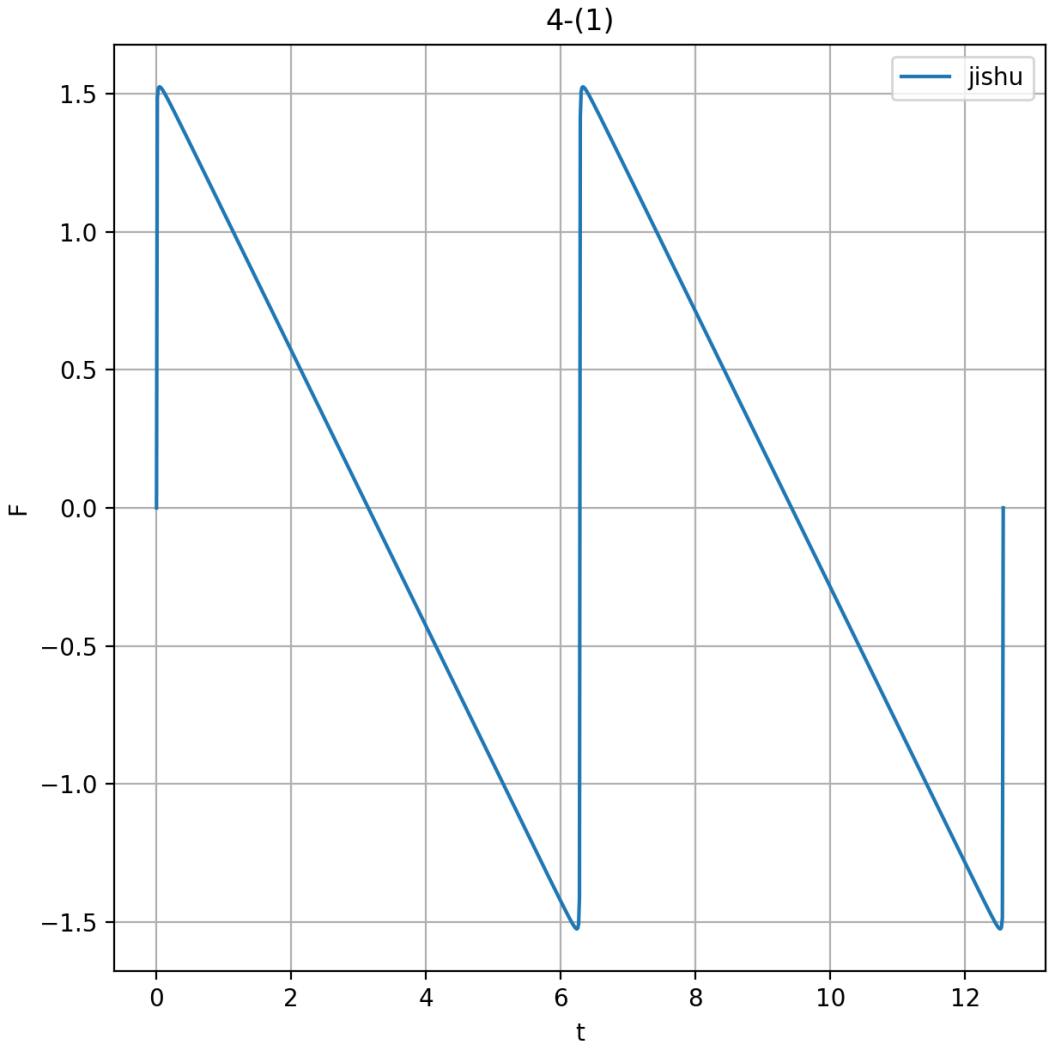

3
2
1
0
−1
−2
−3
0
200
400
600
800
1000

自相关函数

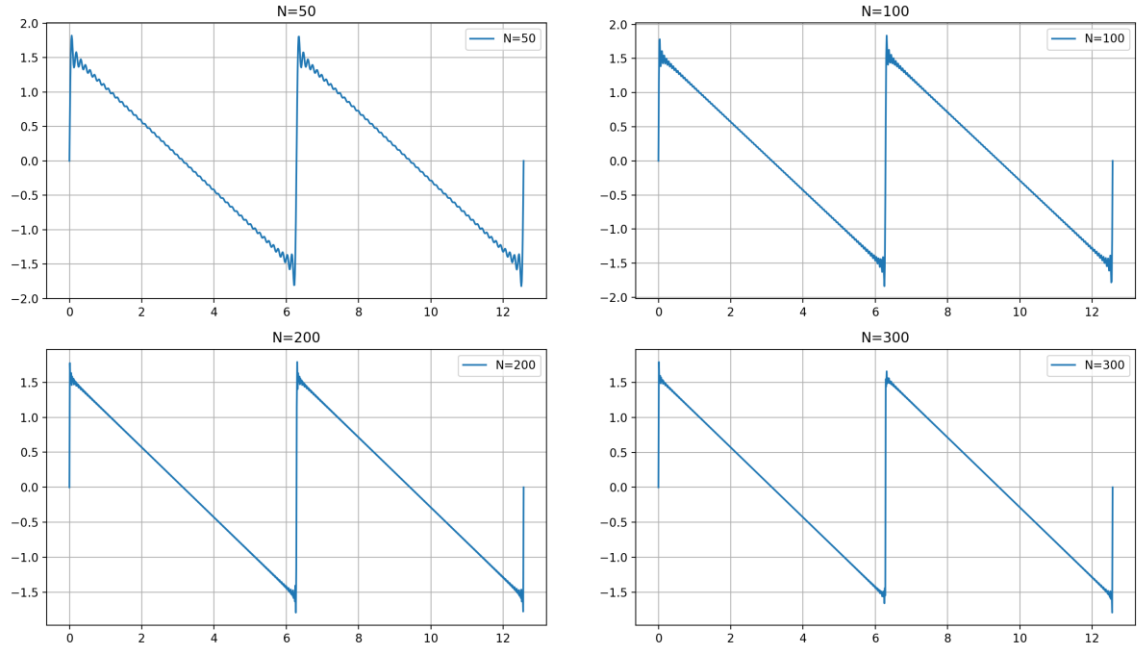

Gaussian White Noise
800
600
400
200
0
y
0
250
500
750
1000
1250
1500
1750
2000
T

4-（1）

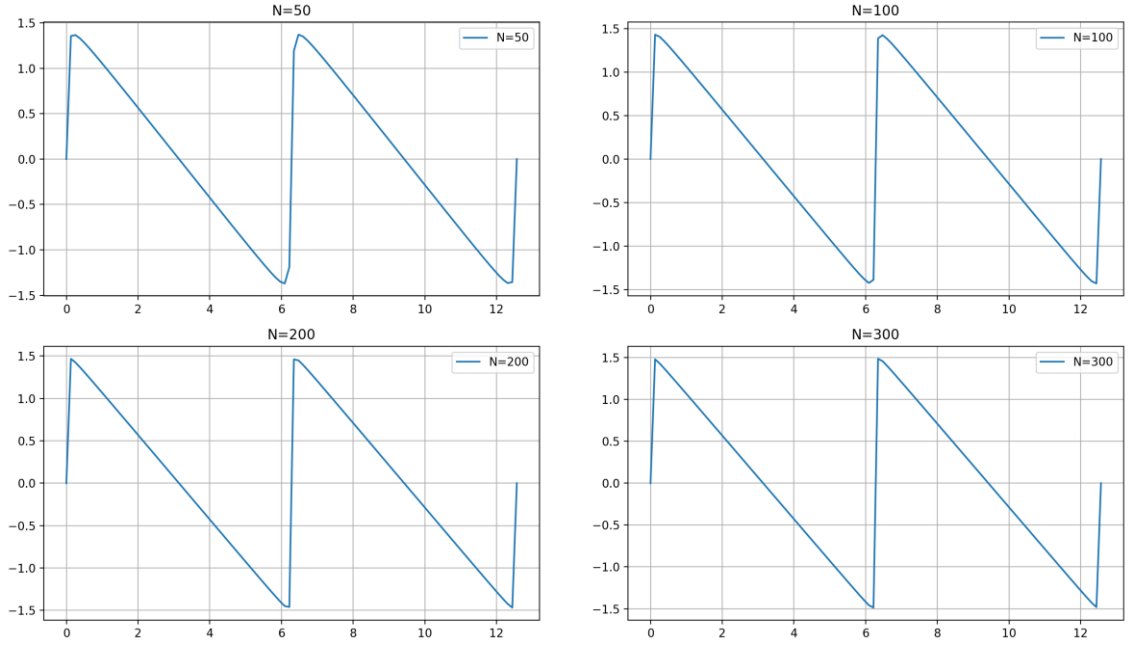
4-(1)



4-(3)



4-(4)



三、实验结果分析

根据学习知识，可以算出前两道题的结果，对比结果和绘制出即可确定绘制图像的结果是否正确。

其中第四题实际上是傅里叶变换的相关可视化，通过不同的波的叠加，深入理解傅里叶变换的相关内容。

实验总结：

心得：

加深了对卷积、单位阶跃响应、单位冲击响应的理解，进一步提高了用编程语言来绘图的相关能力，对于常见的信号，如门函数、阶跃函数等也有了新的认识，加深了对傅里叶级数的理解。

碰到的问题和解决方法：

1、 对处理题目需要的信号库不清楚，查找资料得出相关库的使用方法。

2、 对 python 相关的数学符号，如指数函数、三角函数的调用还不够熟练，对 python 绘图的相关库操作掌握的也不是很熟练。解决方法：网络查找相关资料，主要针对 matplotlib 和 numpy 库的相关功能的搜索。

3、 第 4 题最后一问循环有点绕进去，再多看题去理解意思才得出结果。

参考文献

1、 numpy 库的相关使用：

https://blog.csdn.net/m0_74344139/article/details/134842295

2、 matplotilb 库的相关使用：

https://www.matplotlib.net/stable/index.html

3、 python 与信号有关的库以及使用：

https://blog.csdn.net/m0_52387305/article/details/123440801